

resolved Promise is fulfilled. An error case of a Promise is the rejected state.

Using the `readFile()` function, we can see how the Promise code looks.

```
1. #!/usr/local/bin/node
2. "use strict";
3. console.log('first line');
4. let fs = require("fs");
5. function readFile(filename) {
6.   let p = new Promise(function(resolve, reject) {
7.     fs.readFile(filename, function(err, contents) {
8.       if (err) {
9.         reject(err); // error case
10.      }
11.      resolve(contents);
12.    });
13.  });
14.  return p;
15. }
16. let promise = readFile("readme.txt");
17.
18. promise.then(fulfill, reject);
19.
20. function fulfill (contents) {
21.   console.log(contents.toString());
22. }
23.
24. function reject (err) {
25.   console.log(err.message)
26. }
27. console.log('last line');
```

The output of the above program is shown below:

```
first line
last line
README file contents
```

Line 16: This invoked the function `readFile()`, which returns a Promise.

Line 18: After the file contents are read, the Promise comes to the resolved state. For a successful file read, `fulfill()` gets executed. For an error scenario like the file not existing, 'reject' gets executed.

One of the advantages of Promise is chaining, which is done by appending multiple `.then()` functions. You can also have one error handling function using `.catch()`. Line 18 can be changed to what's shown below:

```
promise.then(fulfill).then(process).then(processFurther);
```

...or, what follows:

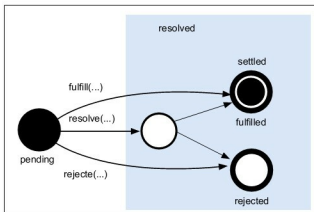


Figure 1: Promise states diagram

```
promise.then(fulfill).then(process).catch(reject);
```

Promise.all() and Promise.race()

There are two more methods that Promise provides. These two are used when you have to iterate through multiple Promises (async operations). Instances of it could be useful when you have to gather content from multiple sources/APIs.

```
Promise.all([p1, p2, p3]).then(function(value) {
  console.log(value);
}), function(reason) {
  console.log(reason)
});
```

`Promise.all()` takes multiple Promises as an array. The first function of `then` is executed when all Promises (p1, p2 and p3) are fulfilled. Even if one fails, the second function is called for error handling.

`Promise.race()` is similar, except that the parent Promise will be fulfilled even if only one of the three Promises are fulfilled. This could be used when fetching dictionary data from multiple sources. The result from whichever source is fetched faster, and is good to display. This method can also be used to timeout after predefined milliseconds.

Promises with arrow functions

In previous articles in this series, we covered another feature of ES6 called arrow functions. They come handy when used in combination with Promises.

The earlier program, from Lines 8 to 16, can be modified to use anonymous functions, as follows:

```
promise.then(function (contents) {
  console.log(contents.toString());
}), function (err) {
  console.log(err.message)
});
This can further be enhanced to:
promise.then((contents) => {
```